

Módulo B2B Purchase Order

Documento Funcional y Arquitectónico

Magento_PurchaseOrder

Módulo	Plataforma	Fecha
vendor/magento/module-purchase-order	Adobe Commerce B2B	Abril 2026

Documento Funcional: Módulo B2B Purchase Order (Magento_PurchaseOrder)

1. Propósito del Módulo

El módulo `Magento_PurchaseOrder` (ubicado en `vendor/magento/module-purchase-order`) implementa la funcionalidad de **Órdenes de Compra (Purchase Orders / PO)** para entornos B2B de Adobe Commerce.

Su objetivo es permitir a los empleados de una empresa generar peticiones de compra formales que, en lugar de convertirse directamente en pedidos, pasan por un **flujo de aprobación** antes de ejecutarse. Esto soporta procesos de control de gasto, jerarquías de autorización y auditoría internos de empresas corporativas.

2. Dependencias del Módulo

El módulo se integra profundamente con el ecosistema B2B y algunos módulos core:

Módulo	Rol
<code>Magento_Company</code>	Gestión de empresas y roles/usuarios corporativos
<code>Magento_Quote</code>	El PO se basa en un Quote (carrito) como origen de datos
<code>Magento_Sales</code>	Generación del Order final una vez aprobado el PO
<code>Magento_NegotiableQuote</code>	Integración: un PO puede originarse desde un presupuesto negociado
<code>Magento_CompanyCredit</code>	Integración con métodos de pago diferido tipo crédito de empresa
<code>Magento_SalesSequence</code>	Generación de increment_id secuenciales para los POs
<code>Magento_MessageQueue</code> / <code>Magento_Amqp</code>	Procesamiento asíncrono de órdenes tras aprobación
<code>Magento_Checkout</code>	Intercepta el proceso de checkout para redirigir a PO

3. Esquema de Base de Datos

3.1 Tablas Creadas (`db_schema.xml`)

`PURCHASE_ORDER` — ENTIDAD PRINCIPAL

Campo	Tipo	Descripción
<code>entity_id</code>	INT (PK, AI)	ID interno del PO
<code>increment_id</code>	VARCHAR(50)	ID legible secuencial (ej. PO-000001)
<code>quote_id</code>	INT (UNIQUE FK)	Referencia al Quote de Magento (1:1)
<code>status</code>	VARCHAR(255)	Estado del PO (ver sección 5)
<code>creator_id</code>	INT (FK → customer_entity)	Cliente que creó el PO
<code>company_id</code>	INT (FK → company, ON DELETE CASCADE)	Empresa a la que pertenece
<code>address_id</code>	INT	ID de dirección de envío
<code>shipping_method</code>	VARCHAR(40)	Método de envío seleccionado
<code>payment_method</code>	VARCHAR(255)	Método de pago seleccionado
<code>grand_total</code>	DECIMAL(12,4)	Total bruto del PO
<code>snapshot</code>	MEDIUMTEXT	Snapshot JSON del Quote en el momento de creación
<code>is_validate</code>	SMALLINT	Flag de validación interna
<code>order_id</code>	INT	ID del Order generado (null hasta aprobación final)
<code>order_increment_id</code>	VARCHAR(50)	Increment ID del Order generado
<code>auto_approved</code>	BOOLEAN	¿Fue aprobado automáticamente?
<code>created_at</code>	TIMESTAMP	Fecha de creación
<code>updated_at</code>	TIMESTAMP	Fecha de última actualización

PURCHASE_ORDER_APPROVED_BY — APROBADORES

Campo	Tipo	Descripción
<code>purchase_order_id</code>	INT (FK → purchase_order, CASCADE)	PO aprobado
<code>customer_id</code>	INT (FK → customer_entity, CASCADE)	Cliente que aprobó

> Clave primaria compuesta. Permite múltiples aprobadores por PO.

PURCHASE_ORDER_LOG — AUDITORÍA/HISTORIA

Campo	Tipo	Descripción
<code>id</code>	INT (PK, AI)	ID del registro
<code>request_id</code>	INT (FK → purchase_order, CASCADE)	PO al que pertenece
<code>request_log</code>	TEXT	Mensaje del log (con parámetros en JSON)
<code>activity_type</code>	TEXT	Tipo de actividad: <code>submit</code> , <code>approve</code> , <code>reject</code> , <code>cancel</code> , <code>place_order</code> , <code>auto_approve</code> , <code>place_order_fail</code>
<code>created_at</code>	TIMESTAMP	Moment del evento
<code>owner_id</code>	INT	ID del actor que realizó la acción (null = sistema)

`PURCHASE_ORDER_COMMENT` — COMENTARIOS

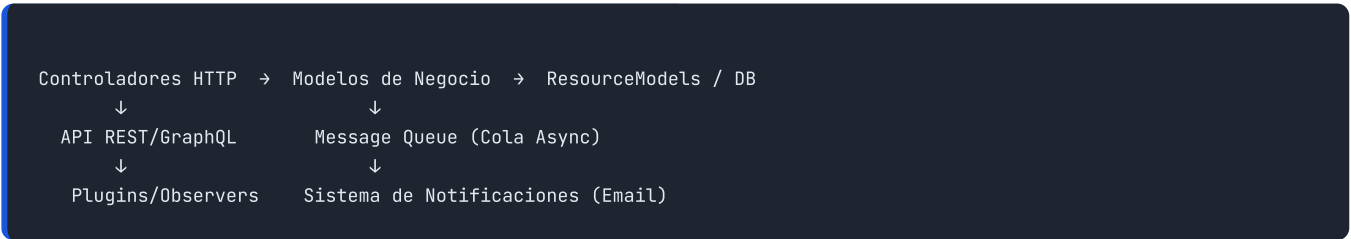
Campo	Tipo	Descripción
<code>entity_id</code>	INT (PK, AI)	ID del comentario
<code>purchase_order_id</code>	INT (FK → purchase_order, CASCADE)	PO relacionado
<code>creator_id</code>	INT	Autor del comentario
<code>comment</code>	TEXT	Texto del comentario
<code>created_at</code>	TIMESTAMP	Fecha

`PURCHASE_ORDER_COMPANY_CONFIG` — CONFIGURACIÓN POR EMPRESA

Campo	Tipo	Descripción
<code>company_entity_id</code>	INT (PK, FK → company, CASCADE)	ID de empresa
<code>is_purchase_order_enabled</code>	SMALLINT	¿PO habilitado para esta empresa?

4. Arquitectura del Módulo

4.1 Capas Arquitectónicas



4.2 Interfaces Públicas (API Layer — `Api/`)

- `PurchaseOrderInterface` — Datos del PO
- `PurchaseOrderRepositoryInterface` — CRUD del PO
- `PurchaseOrderManagementInterface` — Operaciones de ciclo de vida
- `PurchaseOrderPaymentInformationManagementInterface` — Información de pago para aprobados pendientes de pago

4.3 Clases Modelo Principales (`Model/`)

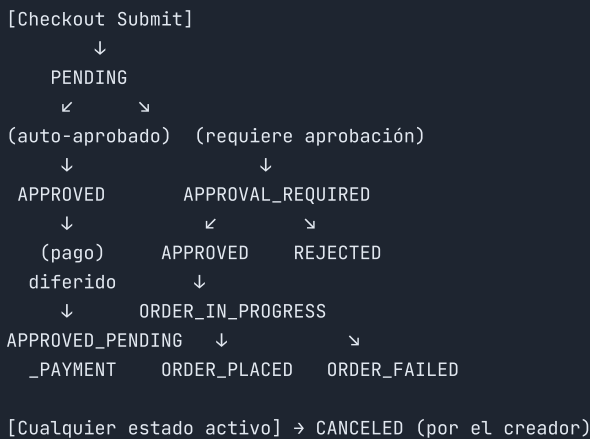
Clase	Responsabilidad
PurchaseOrder	Entidad del PO, serialización del snapshot del Quote
PurchaseOrderProcessor	Creación del PO desde un Quote (checkout)
PurchaseOrderManagement	Ciclo de vida: approve, reject, cancel, createSalesOrder
PurchaseOrderRepository	Persistencia CRUD del PO
PurchaseOrderBulkManagement	Operaciones masivas sobre múltiples POs
PurchaseOrderQuoteConverter	Convierte un Quote a array y viceversa para el snapshot
Config	Lógica de activación por website y por empresa
PurchaseOrderLog / PurchaseOrderLogRepository	Entidad y persistencia del log de auditoría
Comment / CommentManagement / CommentRepository	Sistema de comentarios en el PO
PriceFormatter	Formateo de precios en contexto B2B

5. Máquina de Estados (Status Machine)

El PO puede encontrarse en uno de los siguientes estados:

Constante	Valor	Descripción
STATUS_PENDING	pending	Recién creado, pendiente de evaluación
STATUS_APPROVAL_REQUIRED	approval_required	Requiere aprobación manual de un superior
STATUS_APPROVED	approved	Aprobado — el sistema procederá a crear el Order
STATUS_APPROVED_PENDING_PAYMENT	approved_pending_payment	Aprobado pero con método de pago diferido pendiente
STATUS_ORDER_IN_PROGRESS	order_in_progress	Se está ejecutando la creación del Order (transitorio)
STATUS_ORDER_PLACED	order_placed	Order creado correctamente — PO finalizado
STATUS_ORDER_FAILED	order_failed	El intento de crear el Order falló
STATUS_REJECTED	rejected	Rechazado por un aprobador
STATUS_CANCELED	cancelled	Cancelado por el creador

Diagrama de Transiciones



6. Flujo de Creación de un Purchase Order

6.1 Trigger: Checkout

El módulo intercepta el proceso de checkout estándar mediante:

- **Plugin** `Magento\PurchaseOrder\Plugin\Checkout\Controller\Index\IndexPlugin` — Redirige al checkout propio de PO si la empresa tiene POs habilitados.
- **Plugin** `Magento\PurchaseOrder\Plugin\Quote\Model\QuoteManagementPlugin` — Intercepta `CartManagement::placeOrder()` para redirigir a la creación de PO en lugar de crear un Order directamente.

6.2 Creación del PO: `PurchaseOrderProcessor::createPurchaseOrder()`

1. Obtiene el método de envío del Quote activo.
2. Crea una nueva entidad `PurchaseOrder` con:
 - `quote_id`, `company_id`, `creator_id`, `status = PENDING`
 - Snapshot serializado del Quote (JSON completo de artículos, precios, dirección, etc.)
 - `shipping_method`, `payment_method`, `grand_total`
1. **Bloquea los precios** del Quote: establece `custom_price` y `original_custom_price` en cada item para que los precios no cambien aunque el catálogo varíe.
2. Marca el Quote como **inactivo** (`is_active = false`) para evitar modificaciones.
3. Guarda el PO.
4. Actualiza el log del NegotiableQuote si existe.
5. Registra en el log de auditoría la acción `submit`.
6. Dispara el proceso de evaluación de aprobación automática.

6.3 Post-Creación: Procesadores de Aprobación

Inmediatamente después de crear el PO, se ejecuta el **ApprovalProcessorComposite** (patrón Composite con prioridad):

Procesadores (por prioridad):

1. `AdminApprovalProcessor` (prioridad 10) — Verifica si el creador tiene el permiso de empresa `Magento_PurchaseOrder::autoapprove_purchase_order`. Si sí, aprueba automáticamente.
2. `ManagerApprovalProcessor` (prioridad 20) — Verifica si se requiere aprobación del gestor según las reglas de la empresa. Si no hay reglas que lo requieran, aprueba automáticamente.

Si ningún procesador aprueba el PO, éste queda en `APPROVAL_REQUIRED` y se notifica al manager/superior.

7. Flujo de Aprobación

7.1 Aprobación Manual: `PurchaseOrderManagement::approvePurchaseOrder()`

Validación previa: El PO debe estar en `PENDING` o `APPROVAL_REQUIRED`.

Lógica:

1. Si el aprobador es el mismo que el creador o `$actorId = null` → es auto-aprobación.
2. Evalúa si el método de pago es diferido (`DeferredPaymentStrategy`):
 - Si es diferido → estado `APPROVED_PENDING_PAYMENT`, notificación de pago pendiente.
 - Si no → estado `APPROVED`, notificación de aprobado.
1. Guarda el aprobador en tabla `purchase_order_approved_by`.
2. **Publica en la cola de mensajes** `purchaseorder.toorder` el ID del PO para procesamiento asíncrono.

7.2 Rechazo: `PurchaseOrderManagement::rejectPurchaseOrder()`

Validación: PO en `PENDING` o `APPROVAL_REQUIRED`.

- Estado → `REJECTED`
- Notificación al creador.
- Cierra el NegotiableQuote relacionado si existe.

7.3 Cancelación: `PurchaseOrderManagement::cancelPurchaseOrder()`

Puede cancelar el creador en estados: `PENDING`, `APPROVAL_REQUIRED`.

- Estado → `CANCELED`
- Log de auditoría.
- Cierra el NegotiableQuote relacionado si existe.

8. Conversión del PO a Order (Colocación)

8.1 Cola de Mensajes

- **Topic:** `purchaseorder.toorder`
- **Consumer:** `Magento\PurchaseOrder\Model\QueueConsumer` → llama a `PurchaseOrderManagement::createSalesOrder()`

8.2 `PurchaseOrderManagement::createSalesOrder()`

1. Valida que el PO esté en estado `APPROVED` → cambia a `ORDER_IN_PROGRESS`.
2. Obtiene el Quote asociado por `quote_id`.
3. Reactiva el Quote (`is_active = true`).
4. Llama a `CartManagement::placeOrder()` para generar el Order de Magento.
5. Actualiza el PO: `order_id`, `order_increment_id`, estado → `ORDER_PLACED`.
6. Envía el email de confirmación del pedido.
7. Actualiza el estado del NegotiableQuote a `ordered` si aplica.

En caso de error: Estado → `ORDER_FAILED`, notificación al creador, log de error.

9. Control de Acceso (ACL por Roles de Empresa)

Definido en `etc/company_acl.xml`, los permisos se configuran por **roles de empresa** (no por roles de administrador):

Recurso ACL	Título	Descripción
<code>Magento_PurchaseOrder::all</code>	Order Approvals	Nodo padre de todos los permisos
<code>Magento_PurchaseOrder::view_purchase_orders</code>	View my Purchase Orders	Ver los propios POs
<code>Magento_PurchaseOrder::view_purchase_orders_for_subordinates</code>	View for subordinates	Ver POs de subordinados
<code>Magento_PurchaseOrder::view_purchase_orders_for_company</code>	View for all company	Ver todos los POs de la empresa
<code>Magento_PurchaseOrder::autoapprove_purchase_order</code>	Auto-approve POs	Los POs de este rol se auto-aprueban

Autorizaciones por Acción (`Model/Customer/Authorization/`)

Acción	Clase validadora
<code>placeorder</code>	<code>PlaceOrder</code>
<code>approve</code> , <code>reject</code>	<code>ApproveRejectPurchaseOrder</code>
<code>cancel</code>	<code>CancelPurchaseOrder</code>
<code>view</code>	<code>ViewPurchaseOrder</code>

10. Sistema de Activación (Config)

Activación a dos niveles (`Model/Config.php`):

1. **Website level** — Config en `b2b/website_configuration/purchaseorder_enabled` (path de sistema).
2. **Company level** — Cada empresa tiene en `purchase_order_company_config.is_purchase_order_enabled` su propio flag.

El PO solo está disponible cuando **ambos niveles están habilitados**.

Adicionalmente, el módulo B2B de empresa (`Magento_Company`) debe estar activo en el website.

11. Sistema de Notificaciones por Email

10 eventos de email configurados en `etc/config.xml` y `etc/email_templates.xml` :

Evento	Destinatario	Cuándo se envía
<code>purchase_order_approval_request</code>	Manager/Approver	Al solicitar aprobación
<code>purchase_order_approval_required</code>	Creador del PO	Para informarle que se requiere aprobación
<code>purchase_order_approval_required_payment_details</code>	Creador del PO	Aprobación requerida + detalles de pago pendiente
<code>purchase_order_auto_approved</code>	Creador del PO	Auto-aprobado
<code>purchase_order_auto_approved_payment_details</code>	Creador del PO	Auto-aprobado con pago pendiente
<code>purchase_order_approved</code>	Creador del PO	Aprobado manualmente
<code>purchase_order_approved_payment_details</code>	Creador del PO	Aprobado con pago pendiente
<code>purchase_order_rejected</code>	Creador del PO	Rechazado
<code>purchase_order_comment_added</code>	Participantes conversación	Nuevo comentario añadido
<code>purchase_order_order_place_failed</code>	Creador del PO	Error al generar el Order

Los destinatarios se resuelven mediante **Recipient Resolvers** (inyectados vía DI):

- `PurchaseOrderCreator` — El creador del PO.
- `CreatorStructureParent` — El superior jerárquico del creador.
- `ConversationParticipants` — Todos los que han comentado en el PO.

12. Plugins del Módulo

Plugin	Intercepta	Función
<code>QuoteManagementPlugin</code>	<code>CartManagement::placeOrder()</code>	Redirige a creación de PO si aplica
<code>QuoteRepositoryPlugin</code>	<code>CartRepository</code>	Filtros sobre quotes relacionados con POs
<code>QuotePlugin</code>	<code>Quote::beforeSave()</code>	Lógica extra antes de guardar el quote
<code>AddressPlugin</code>	<code>Quote\Address</code>	Gestión de atributos de dirección
<code>SessionPlugin</code>	<code>Checkout\Session</code>	Guarda el PO ID en sesión post-checkout
<code>IndexPlugin</code>	<code>Checkout\Controller\Index</code>	Redirige el checkout a flujo PO
<code>CompanyRepository</code>	<code>CompanyRepositoryInterface::save()</code>	Guarda config de PO al guardar empresa
<code>DataProvider</code>	<code>Company\Model\DataProvider</code>	Inyecta config de PO en formulario de empresa
<code>UpdateCouponUsagesPlugin</code>	<code>SalesRule\UpdateCouponUsages</code>	Gestiona usos de cupones con POs
<code>CompanyCreditPlugin</code>	<code>CompanyCreditPaymentConfigProvider</code>	Visibilidad del pago por crédito en PO
<code>ExpiredQuotesCollectionFilter</code>	Colección de quotes expirados	Excluye quotes vinculados a POs activos de la expiración
<code>PermissionCheckPlugin</code>	<code>AsyncOrder\AsyncPaymentPublisher</code>	Verifica permisos en órdenes asíncronas con PO

13. API REST

Endpoints definidos en `etc/webapi.xml`:

Método	Ruta	Descripción
GET	<code>/V1/purchase-orders/{purchaseOrderId}</code>	Obtiene un PO por ID
GET	<code>/V1/purchase-orders</code>	Lista los POs del cliente actual
POST	<code>/V1/purchase-orders/{id}/approve</code>	Aprueba el PO
POST	<code>/V1/purchase-orders/{id}/reject</code>	Rechaza el PO
POST	<code>/V1/purchase-orders/{id}/cancel</code>	Cancela el PO
POST	<code>/V1/purchase-orders/{id}/place-order</code>	Coloca la orden del PO aprobado
POST	<code>/V1/purchase-orders/{id}/items</code>	Añade ítems al PO
GET/POST	<code>/V1/purchase-orders/{id}/comments</code>	Gestiona comentarios

14. GraphQL

El módulo expone la operación de PO vía GraphQL definida en `etc/graphql/` (schema parcial), permitiendo operaciones de consulta de POs en proyectos headless/PWA Studio.

15. Log de Auditoría

Implementación: `PurchaseOrder\LogManagement::logAction()`

Cada acción relevante genera un registro en `purchase_order_log`:

- `submit` — Creación del PO
- `auto_approve` — Aprobación automática
- `approve` — Aprobación manual
- `reject` — Rechazo
- `cancel` — Cancelación
- `place_order` — Order creado correctamente (con `order_increment_id`)
- `place_order_fail` — Fallo al crear el Order (con `error_message`)

Los mensajes de log se configuran en `di.xml` bajo `Block\PurchaseOrder\History` y son translatable.

16. Increment ID

El PO usa el sistema de secuencias de Magento (`Magento_SalesSequence`). El entity type `purchase_order` se registra en el pool de entidades y el increment ID se genera automáticamente al primer guardado en `PurchaseOrder::beforeSave()`.

17. Patrones Arquitectónicos Clave

Patrón	Uso en el módulo
Repository Pattern	<code>PurchaseOrderRepository</code> , <code>CommentRepository</code> , <code>PurchaseOrderLogRepository</code>
Interface Segregation	Interfaces separadas en <code>Api/</code> y <code>Api/Data/</code>
Composite Pattern	<code>ApprovalsComposite</code> para la cadena de procesadores de aprobación; <code>Validator\Composite</code>
Strategy Pattern	<code>DeferredPaymentStrategyInterface</code> para detectar pagos diferidos
Observer/Plugin	12 plugins para interceptar flujos de checkout, quote y empresa
Message Queue	Desacoplamiento del <code>approve</code> con el <code>placeOrder</code> (async via AMQP)
Snapshot	El Quote se serializa en JSON en el momento del PO para preservar precios históricos
ACL by Company Role	<code>company_acl.xml</code> define permisos que los administradores de empresa asignan a roles
Virtual Types (DI)	<code>approvalProcessorComposite</code> , <code>myPurchaseOrderCollection</code> , <code>placePoValidatorComposite</code>

18. Resumen para Implementación de Módulo Simplificado

Para crear un módulo simplificado con la misma funcionalidad core, los componentes esenciales son:

Elementos Mínimos Necesarios

1. **Tabla** `purchase_order` — Con los campos: `entity_id`, `increment_id`, `quote_id`, `status`, `creator_id`, `company_id`, `grand_total`, `snapshot`, `created_at`.
2. **Tabla** `purchase_order_log` — Para auditoría.
3. **Modelo + ResourceModel + Collection** — Patrón estándar Magento.
4. **Statuses** — Al menos: `pending`, `approved`, `rejected`, `canceled`, `order_placed`.
5. **PurchaseOrderProcessor** — Intercepta el checkout y crea la entidad PO.
6. **PurchaseOrderManagement** — `approve()`, `reject()`, `cancel()`, `createOrder()`.
7. **Snapshot del Quote** — Serializar el cart al crear el PO para preservar precios.
8. **Plugins de Checkout** — Interceptar `CartManagement::placeOrder()` y `Checkout\Session`.
9. **Controladores** — `Index`, `View`, `Approve`, `Reject`, `Cancel`, `PlaceOrder`.
10. **ACL de empresa** — `company_acl.xml` con permisos mínimos de view/approve.
11. **Config** — Activación a nivel de website + empresa.
12. **Email** — Al menos: solicitud de aprobación, aprobado, rechazado.

Simplificaciones Posibles

- Eliminar soporte a **NegotiableQuote**.
- Eliminar **pagos diferidos** (`APPROVED_PENDING_PAYMENT`).
- Usar aprobación **directa** sin cola de mensajes (sin AMQP).
- Simplificar ACL a un flag único "puede aprobar".
- Eliminar **GraphQL** si no es necesario.
- Eliminar operaciones **bulk**.